TrustVault: AI Prompts and Development Notes

PROMPTS | Snovasys Fresher Hiring Challenge 2026

1. Why this file

I built TrustVault with help from AI tools, and I do not want to hide that. Using AI is normal now. What matters is whether I understand the code, check it, and can explain it. This file lists the prompts I used, why I used them and what I did with the answers.

Please note that the prompts below are written in a clean form and they show the way I worked on each task. They are not a copy of every conversation word by word. I did not accept every answer as it was. Some answers had mistakes, and sometimes I had to ask again or change the code myself.

2. Context I gave to the AI

At the start of a session I pasted a short description so that the AI would not guess.

Item	Detail
Project	TrustVault, secure document sharing with an audit trail
Frontend	React and Vite, JavaScript
Backend	Node.js and Express
Database	MySQL with users, documents, document_permissions and audit_logs
Login	JWT, passwords hashed with bcryptjs
Uploads	Multer, private local folder, no cloud storage
Testing	Jest and Supertest
Environment	Windows, VS Code, PowerShell, Git and GitHub

I also said that I prefer simple explanations. This helped a lot, because I could ask why a line is there and get an answer that I could repeat in my own words.

3. How I worked

1. I asked for a plan first, not code, so that I understand the idea before touching the project.
2. I gave real context, like the actual file, the real error and the existing schema. General questions gave general answers, and many of them were wrong for my project.
3. I asked for small changes that keep the working code, and not full rewrites.
4. I read the code before pasting it. If I could not explain a line, I asked about it again.
5. I ran it and tried to break it. I tested the normal path and the forbidden path.
6. I committed only after it worked, with a message about what changed.

4. The prompts

A. Planning and design

Prompt 1: Understanding the requirements

> *Act as a senior software engineer and help me build TrustVault, a secure document-sharing web application with a real audit trail. I prefer beginner-friendly explanations. Use React with Vite for the frontend, Node.js with Express for the backend, and MySQL for the database. Identify the main features, database entities, API responsibilities, security requirements, and implementation phases. Keep the plan practical for a hiring challenge.*

Why I asked: I had the challenge brief but no clear order of work.

What I did with the answer: I used it as a roadmap: foundation, login, documents, permissions, audit and search, tests, documents. I removed ideas that were too big for the time I had.

How I checked it: I compared the plan with the challenge requirements line by line, so that nothing required was missing.

Prompt 2: Designing the database

> *Help me design a MySQL database for TrustVault. Include users, documents, document_permissions, and audit_logs. Explain the relationships, primary keys, foreign keys, ownership, sharing permissions, and audit fields. Provide SQL that is consistent with the existing project schema, and identify constraints needed to maintain data integrity.*

Why I asked: A weak database design makes permission checks messy later, so I wanted it right at the start.

What I did with the answer: I kept four tables. I made sure there is only one permission row for a user and a document, and that audit rows stay even if a user or document is removed.

How I checked it: I created the tables in MySQL, inserted test rows and checked that duplicates and wrong references were rejected. SQL from AI must always be compared with the real schema before use.

B. Login and permissions

Prompt 3: JWT login

> *Review my existing Node.js and Express backend. Help me implement JWT authentication middleware and a current-user endpoint. Use bcryptjs for password hashing and retrieve the user's current role from MySQL when required. Preserve existing routes and exports. Explain every change and show how to test authenticated and unauthenticated requests.*

Why I asked: Login was my first real security feature and I wanted to understand tokens and not just copy them.

What I did with the answer: I added the middleware and the current-user route. I made the role come from the database and not only from the token.

How I checked it: I called protected routes with no token, a wrong token and a correct token, and checked that the status codes were different.

Prompt 4: Document permissions

> *Help me secure my existing TrustVault document routes. A document owner or a user with an explicitly granted permission should be able to access a protected document. Other users must receive an appropriate HTTP 403 response. Check document ownership and permissions on the backend, prevent cross-user read and download access, and avoid exposing private file paths. Do not rewrite unrelated working code.*

Why I asked: This is the heart of the project. Without it, TrustVault is just a file uploader with a login page.

What I did with the answer: I added the ownership and permission check on the read and download routes, and kept the internal file paths out of the responses.

How I checked it: I logged in as two users and tried to open each other's documents by ID. The denied attempts appear in the audit log.

Prompt 5: Admin routes

> *Review the TrustVault administrative audit-log endpoint. Ensure that unauthenticated requests are rejected and users without the required administrator role cannot access the logs. Follow the project's existing authentication and role-checking patterns. Suggest tests for unauthorized access and explain the expected HTTP status codes.*

Why I asked: The audit log is sensitive because it shows who accessed what.

What I did with the answer: I added a role check after the login check and used the same pattern as my other routes.

How I checked it: I tested with no user and with a normal viewer, which must be refused, and with an admin, which must work.

C. Documents, sharing and audit

Prompt 6: Uploads and downloads

> *Help me implement secure document uploads and downloads using Express and Multer. Use private local filesystem storage, allow only the configured PDF, TXT, JPEG, and PNG formats, and enforce a maximum file size of 5 MB per file. Store document metadata in MySQL. Ensure that downloads pass authorization checks before a file is sent, and explain how to handle invalid files and missing files safely.*

Why I asked: File handling is where many beginner projects become unsafe, so I wanted to see the safe way.

What I did with the answer: I set the size and type limits, saved the files with random UUID names and kept only the file details in the database.

How I checked it: I tried a wrong file type and a big file, which must be rejected, and a download where the file was missing from the folder. The real configuration has to be checked against the code.

Prompt 7: Giving and removing access

> *Help me implement document permission granting and revocation in TrustVault. Only authorized document owners should be able to manage sharing permissions. Ensure that permission changes are validated, stored in MySQL, and reflected in subsequent authorization checks. Suggest test cases for unauthorized permission changes and revoked access.*

Why I asked: Sharing is useless if removing access does not really work.

What I did with the answer: I made grant and revoke so that the permission is checked again on every request.

How I checked it: I gave access, confirmed the other user could read, removed it, and confirmed that the next request was denied.

Prompt 8: Audit logging

> *Help me implement audit logging for relevant TrustVault document activities, including uploads, downloads, and denied access attempts. Use the existing audit_logs table and database connection. Record the authenticated user where available, the relevant action, the document identifier where appropriate, and a timestamp. Avoid recording passwords, JWTs, or unnecessary sensitive document contents.*

Why I asked: I wanted a real record in the database and not only messages in the console.

What I did with the answer: I wrote an audit row for each important action, including denied ones, with a result field and small JSON details.

How I checked it: After each test I looked at the table and checked that every action made the expected row and that no password or token was in the details.

D. Frontend

Prompt 9: React screens

> *Help me build a beginner-friendly React and Vite frontend for TrustVault that works with my existing Express backend. Plan login and registration screens, a user dashboard, document lists, upload forms, document-sharing controls, and audit-log screens for authorized administrators. Use reusable components and handle loading states, API errors, empty results, and expired authentication. Do not invent backend endpoints; check the actual API routes before connecting the frontend.*

Why I asked: I am comfortable with HTML and CSS but newer to React. AI tools also tend to invent endpoints that do not exist, so I told it not to.

What I did with the answer: I built the screens in small components and matched each API call with a route that really exists in my backend.

How I checked it: I clicked through every screen with the real backend running and checked the loading, error and empty states.

Prompt 10: Connecting frontend and backend

> *Review my React frontend API calls and Express routes. Help me configure the backend URL, authentication headers, request bodies, and response handling correctly. Troubleshoot CORS errors, failed requests, empty document lists, and incorrect route paths. Give me simple Windows PowerShell commands and browser checks to reproduce and verify each issue.*

Why I asked: Connection problems are confusing because the error shows in the browser but the reason is in the server.

What I did with the answer: I fixed the base URL, attached the token to the requests and set the allowed frontend address in the backend.

How I checked it: I used the network tab of the browser and PowerShell requests to compare what was sent with what the server expected.

E. Testing and review

Prompt 11: Security tests

> *Using Jest and Supertest, help me write integration tests for TrustVault's document security. Test cross-user read denial, cross-user download denial, unauthenticated administrative audit-log access, and insufficient-role access. Reuse the existing application and test setup, avoid weakening authorization to make tests pass, and explain the expected results.*

Why I asked: I wanted proof that the rules work, and not only a feeling that they work.

What I did with the answer: I wrote tests that try forbidden actions and expect 401 or 403.

How I checked it: I ran the suite. A test is only useful if it can fail, so a check that is broken on purpose should turn the test red.

Prompt 12: Finding the reason for failures

> *I will provide the exact error message, relevant source code, and test output from my TrustVault project. Identify the likely cause, explain it in simple language, and suggest the smallest safe correction. Preserve existing working features. Provide the exact Windows PowerShell commands to rerun the affected tests and check the Git diff.*

Why I asked: When I pasted the real error and the real code the answers were much better than when I described the problem in a vague way.

What I did with the answer: I applied the smallest fix and then read the Git diff to confirm that only the planned lines changed.

How I checked it: I ran the affected tests again and then the whole suite to be sure the fix did not break something else.

Prompt 13: Review before commit

> *Review my TrustVault changes for correctness, authorization weaknesses, exposed secrets, inconsistent route exports, and unintended changes. Suggest practical checks using the existing Jest test suite and Git. Do not claim the application is secure merely because tests pass. Help me prepare a clean, meaningful Git commit after verification.*

Why I asked: A second look before commit catches mistakes, especially secrets that were added by accident.

What I did with the answer: I checked the staged files, confirmed that .env, uploads and SQL exports were excluded, and wrote a clear commit message.

How I checked it: I used git status and git diff before each commit.

F. Documentation

Prompt 14: README, THREATS and PROMPTS

> *Help me prepare honest README.md, THREATS.md, and PROMPTS.md files for TrustVault. Document the actual technology stack, architecture, security controls, local file-storage approach, test coverage, setup requirements, known limitations, and future improvements. Clearly distinguish implemented features from planned work. Do not invent deployment URLs, passing test results, or production security guarantees.*

Why I asked: I wanted documents that match the real project, without big claims that a reviewer can catch.

What I did with the answer: I rewrote the drafts in my own words, corrected whatever did not match the code and wrote the limitations openly.

How I checked it: I read each sentence and asked myself if I can show it in the running app or in the code. If not, I removed it or made it softer.

5. Where I did not just trust the AI

AI answers sound confident even when they are wrong. So I kept some habits:

- Invented names. AI can mention routes, columns or packages that are not in my project. I checked each one in my files.
- Shortcuts in authorization. Some suggestions only checked that the user is logged in and not that the user owns or was given the document. I treated every document route as unsafe until it did both checks.
- Passing tests does not mean secure. I told the AI not to say it, and I kept the same care in my documents.

- Secrets. I never pasted real passwords, keys or the JWT secret in a prompt. I used placeholders.
- Big rewrites can delete working code, so I asked for the smallest change and read the Git diff.

6. What the running system shows

These numbers are from my database export of 24 October 2026. They do not promise security, but they show that the features I asked for gave behaviour that I can see.

Audit table	**Count**
Denied document reads	37
Denied document downloads	37
Permission grants (success)	35
Permission revokes (success)	18
Uploads (success)	5
Denied admin downloads	23
All audit records	162

The denied rows matter most to me. They show that forbidden requests were refused and also recorded, which is what Prompts 4 and 8 were for.

7. Which prompt belongs to which part

Prompts	**Part of the project**	**Main check**
1, 2	Plan and database	Tables created and constraints tested
3, 4, 5	Login, permissions, admin role	Requests with no token, wrong user, wrong role
6, 7, 8	Uploads, sharing, audit log	Rejected uploads, removal test, audit rows
9, 10	React and connection	Click through with the real backend
11, 12, 13	Tests, debugging, review	Full test run, git diff and status
14	Documents	Each statement compared with the code

I have not written commit hashes here on purpose. The commit history on GitHub is the correct record of when each change was made.

8. What I learned about prompting

- A specific prompt gives a useful answer. The prompt "fix my login" is weak. A prompt with my middleware and the exact error is much better.

- Telling the AI what not to do, like not rewriting unrelated code and not inventing endpoints, is as important as telling it what to do.
- Asking why after getting code is where the learning happens.
- Asking for test cases and not only code gives safer features.
- Short sessions with one clear goal worked better than very long chats.

9. Rules I followed

1. Understand before using. I do not keep code I cannot explain.
2. Keep working code and make focused changes.
3. Verify by running the tests and using the application.
4. Protect secrets. No passwords, keys or production credentials in prompts or public files.
5. Do not write claims that I cannot prove.
6. Give extra attention to login, permissions, file handling and SQL.
7. AI helps, but I am responsible for the result.

10. Final thoughts

If I had built TrustVault without AI, I would probably have finished fewer features and made more mistakes. If I had accepted everything from AI without thinking, I would have more code but less understanding, and the first question in a demo would show it. The best way for me was in the middle: use AI as a fast helper and a patient teacher, and then do the real work of reading, testing and fixing.

TrustVault is still a learning project. Its features and security claims should be judged by the real source code, the current test results and the running application, and not by any document alone, including this one.